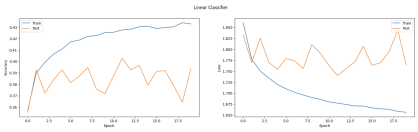
documentation (exercise 2)

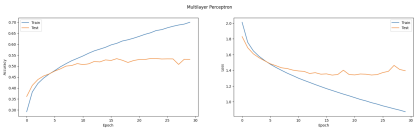
Roberto Sonzini Gobbi: 12505287

1 Experiment 1

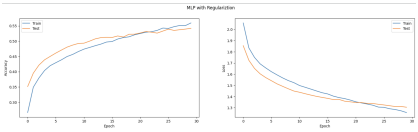
1.1 Question 1



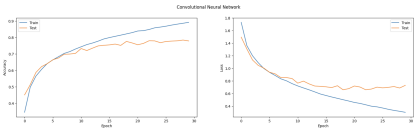
(a) linear classifier



(b) multilayer



(c) multilayer with regularization



(d) convolution

Figure 1: Train vs test error of different models

The models used vary in complexity, which allows them to learn patterns according to their characteristics. Simpler models, such as the linear classifier, struggle to capture complex patterns present in images.

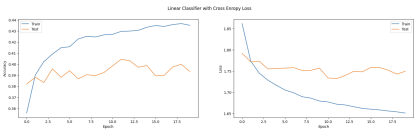
The low accuracy indicates that the model is not able to perform adequate learning for this task. Moving towards more complex models, we observe increased performance, with convolutional networks and fine-tuning providing significantly better results, which is expected in real-world scenarios. This improvement is due to the higher capacity of these models. Convolutional networks, in particular, are well-suited for image recognition as they can capture local patterns and are less sensitive to variations in scale or position of objects within the image.

Looking at the training plots, we notice important characteristics. For models like the multilayer perceptron (MLP) and the linear classifier, the test error stabilizes while the training error continues to decrease. This indicates overfitting. The accuracy remaining constant and the inability to improve further is another evidence that the model is too simple to fully capture the features of the images.

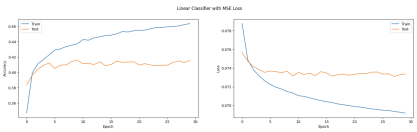
In contrast, the model with regularization (dropout) avoids this overfitting while maintaining similar training accuracy. This occurs because the model's inherent expressive power remains similar, but generalization to unseen images improves.

Finally, the convolutional network shows substantially better performance due to its architecture. However, these gains are not free: training requires more time, and even the convolutional network exhibits slight overfitting.

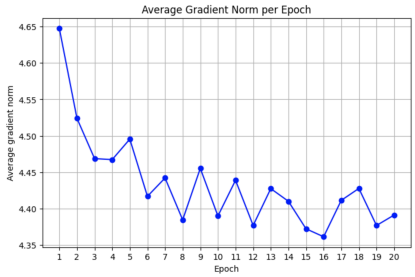
1.2 Question 2



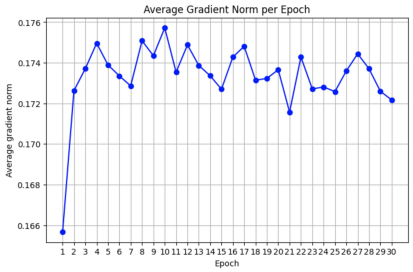
(a) linear classifier with cross entropy



(b) linear classifier with MSE



(c) average gradient norm with cross entropy



(d) average gradient norm with MSE

Figure 2: comparison between cross entropy and MSE

Replacing the cross-entropy loss with mean squared error (MSE) significantly changes both model performance and the training dynamics.

MSE is not well suited for multi-class classification problems because it measures a continuous distance between the predicted vector and the target vector. In classification, however, what we need is a loss that effectively penalizes incorrect probability assignments. As a consequence, MSE produces weaker and less informative gradients.

One particularly clear effect is that the gradient norm remains almost constant throughout training. This happens because MSE does not provide a meaningful notion of distance between the prediction and the correct class in a classification setting. As a result, the optimizer receives very weak update signals.

The low gradient magnitude also requires the use of a much higher learning rate to compensate. Indeed, to obtain reasonable performance with MSE, it was necessary to use very different hyperparameters (learning rate = 0.1, epochs = 30), whereas cross-entropy converges efficiently even with much smaller values.

From the experimental results we observe that, although the final performance is surprisingly similar, the training behavior is fundamentally different:

- MSE requires more epochs to converge;
- the gradient norm is much smaller and almost static while in the cross-entropy it decreases letting us know that we are cnverging to a minimum and learning;

- the model learns more slowly and less effectively than with cross-entropy.

In conclusion, while MSE can sometimes reach acceptable performance, it is not an appropriate loss function for multi-class classification and leads to a considerably less efficient learning process compared to cross-entropy.

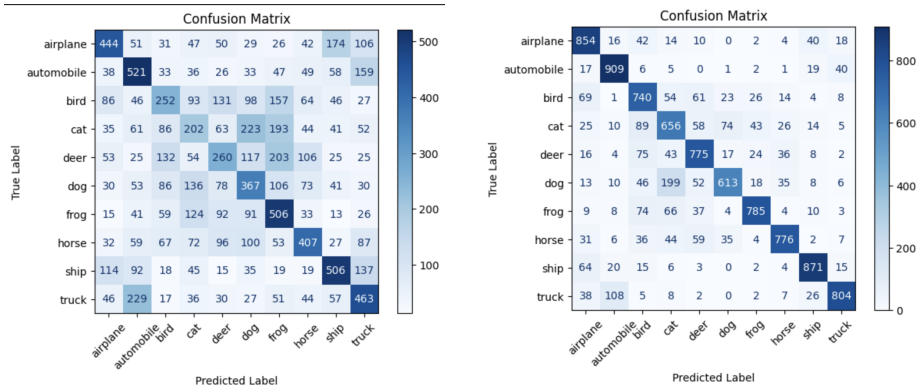
2 Experiment 2

When comparing the confusion matrices of the models, we observe several consistent misclassification patterns. Many errors are related to visual similarity, especially in terms of color and background. For example, airplanes and ships or airplane and birds are often confused because both frequently appear against a blue background, so the model may rely on this cue when making predictions.

Animals like cats and dogs are also commonly misclassified due to their similar colors (gray, brown, black) and because they often occupy only a small portion of the image, making it difficult for the model to extract distinctive features. Vehicle classes such as trucks and automobiles can be mixed up as well, since they share similar color schemes and typically appear in comparable urban environments.

We also observe errors caused by semantic similarity between objects. For instance, dogs and cats, or trucks and cars, are misclassified more often, as shown in the confusion matrices.

Another important factor contributing to misclassification is the presence of noise in the images and the large variation in object scale. Since objects can appear very small or very large depending on the sample, simpler models—especially non-CNN architectures—struggle to handle these variations and cannot extract spatially meaningful patterns, which leads to additional errors.



(a) Confusion Matrix — linear model

(b) Confusion Matrix — CNN

Figure 3: Comparison of the confusion matrices for two different models.



(a) Example 1 (b) Example 2 (c) Example 3

Figure 4: Examples of misclassified images influenced by background or other visual cues.

3 Experiment 3

In a Convolutional Neural Network (CNN), the receptive field of a neuron in a specific layer represents the portion of the original image that influences the neuron's output. In other words, it indicates how much visual context from the input a neuron sees.

As we move deeper into the network, the receptive field typically increases, because neurons in deeper layers aggregate information from larger regions of the original image.

For a layer l in the network, the receptive field can be calculated recursively from the previous layer using the following formula:

$$r_l = r_{l-1} + (k_l - 1) \cdot \prod_{i=1}^{l-1} s_i \tag{1}$$

where:

- r_l is the receptive field of layer l,
- r_{l-1} is the receptive field of the previous layer,
- k_l is the kernel size of layer l,
- s_i is the stride of layer i.

In our case, considering a simple network with three convolutional layers with 3×3 kernels and stride 1, interleaved with two 2×2 max-pooling layers with stride 2, the receptive field for each layer can be approximately calculated as follows:

- Conv 3×3, 32 filters, stride 1: $RF = 3$

- MaxPool 2×2, stride 2: $RF = 4$
- Conv 3×3, 64 filters, stride 1: $RF = 8$
- MaxPool 2×2, stride 2: $RF = 10$
- Conv 3×3, 128 filters, stride 1: $RF = 18$

This demonstrates how the receptive field increases in deeper layers, allowing neurons to aggregate information over larger portions of the original image.